Bayesian Inference for Parameters Associated with the Generalized Extreme Value (GEV) Distribution

Hao

The aim of this work is to implement Bayesian inference for the parameters associated with the GEV distribution for extreme temperature data in New Zealand using RStan.

Importing Library

```
library(rstan)
options(mc.cores = parallel::detectCores())
rstan_options(auto_write = TRUE)
library(tidyverse)
library(loo)
library(evd)
library(gridExtra)
seed = 123456
```

Data Preparation

To prepare a dataset that contains maximum daily average temperatures for each year in 30 different locations in New Zealand.

```
data = read.csv("daily-temperature-1909-2019.csv")
head(data)
```

location_name	date	statistic	temperature	station_name_niwa	agent_number
Auckland	1966-01-01	Maximum	22.2	Auckland Aero	1962
Auckland	1966-01-01	Minimum	13.9	Auckland Aero	1962
Auckland	1966-01-01	Average	18.1	Auckland Aero	1962
Auckland	1966-01-02	Maximum	23.9	Auckland Aero	1962
Auckland	1966-01-02	Minimum	17.1	Auckland Aero	1962
Auckland	1966-01-02	Average	20.5	Auckland Aero	1962

The dataset has 2049470 rows and 6 columns.

The dataset consists of multiple weather metrics recorded daily across 30 distinct locations in New Zealand. Each record includes:

- `location_name`: The name of the location where the measurement was taken.
- `date`: The date of the record.
- `statistic`: Type of temperature statistic - Maximum, Minimum, or Average.
- `temperature`: The recorded temperature.
- `station_name_niwa`: The name of the station that recorded the measurement.
- `agent_number`: A unique identifier for the station.

Since our objective is to analyze the maximum daily average temperatures, we'll start by filtering only those records with the 'Average' statistic. For each location and for each year, find the day with the highest average temperature:

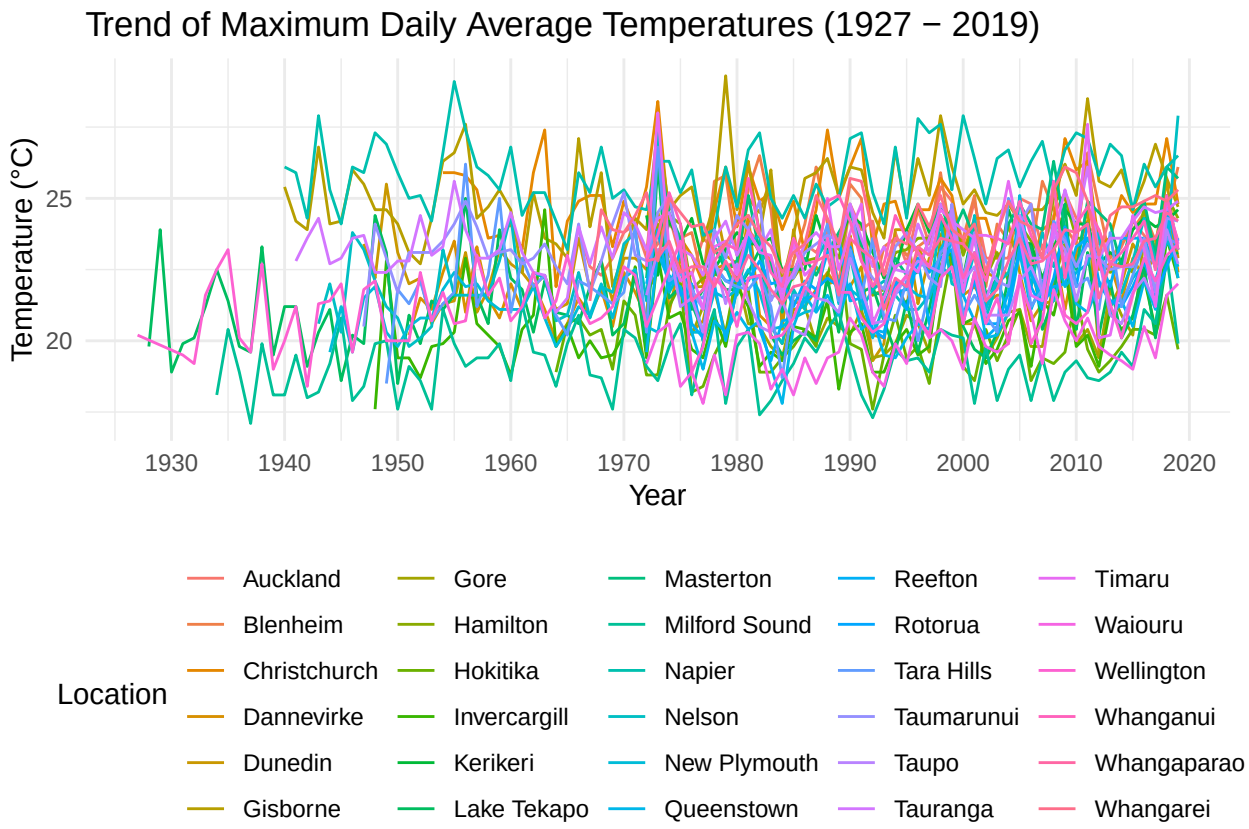
```
avg_temp_data <- data %>%
    filter(statistic == "Average" & !is.na(temperature))

extreme_avg_data <- avg_temp_data %>%
    group_by(location_name, year = year(date)) %>%
    summarize(max_avg_temp = max(temperature, na.rm = TRUE))
```

This `extreme_avg_data` dataframe now holds the maximum daily average temperature for each year and each of the 30 locations.

```
head(extreme_avg_data)
```

location_name	year	max_avg_temp
Auckland	1966	23.9
Auckland	1967	22.6
Auckland	1968	22.3
Auckland	1969	22.9
Auckland	1970	23.8
Auckland	1971	24.1

```
ggplot(extreme_avg_data, aes(x = year, y = max_avg_temp, color = location_name,
    group = location_name)) + geom_line() + labs(title = "Trend of Maximum Daily Average Tempe
    x = "Year", y = "Temperature (°C)", color = "Location") +
    theme_minimal() + theme(legend.position = "bottom") + scale_x_continuous(breaks = scales::
```



There doesn't appear to be any trend in the plot.

Bayesian Inference using Stan

The primary aim of this part is to infer the parameters of the GEV distribution that characterizes the extreme temperatures for each location in New Zealand using Bayesian methods.

The GEV Distribution

We implement the likelihood function in the `functions` block of Stan code, where `gev_lpdf` represents the GEV distribution:

```
functions{
    real gev_lpdf(vector y, real loc, real scale, real shape) {
        vector[rows(y)] z;
        vector[rows(y)] lp;
        real loglike;
        int N;

        N = rows(y);

        if (fabs(shape) > 1e-10) {
            for(i in 1:N){
                z[i] = 1 + shape * (y[i] - loc) / scale;
                lp[i] = (1 + 1 / shape) * log(z[i]) + pow(z[i], -1 / shape);
            }
        loglike = - sum(lp) - N * log(scale);
        } else {
            for (i in 1:N){
                z[i] = (y[i] - loc) / scale;
                lp[i] = z[i] + exp(-z[i]);
            }
            loglike = - sum(lp) - N * log(scale);
        }

        return(loglike);
    }
}
```

Choosing Priors

In the absence of any expert prior information regarding the three parameters of the GEV distribution, we adopt a 'naive' approach and use non–informative, independent priors for these, namely

$$\pi(\mu) \sim \text{Normal}(0, 10),$$
$$\pi(\sigma) \sim \text{Normal}(0, 10),$$
$$\pi(\xi) \sim \text{Normal}(0, 10).$$

This indicates a neutral or weakly informative belief about the parameters before observing the data. The wide standard deviation of 10 for the normal distributions suggests a broad spread, allowing the data to dominate the Bayesian updating process.

We use relatively non-informative priors, specified in the `model` block of Stan code:

```
...
parameters {
    real<lower=-0.5,upper=0.5> shape;
    real<lower=0> scale;
    real loc;
}

model {
    scale ~ normal(0, 10);
    shape ~ normal(0, 10);
    loc ~ normal(0, 10);
...
```

The Posterior Distribution

The posterior distribution represents our updated beliefs about the parameters after seeing the data. In the Stan code, the `model` block implicitly defines the unnormalized posterior distribution:

```
...
y ~ gev(loc, scale, shape);
...
```

Posterior Sampling with RStan

We used the RStan to draw samples from this posterior distribution. The RStan code specifies the GEV likelihood, the normal priors, and the unnormalized posterior distribution as the product of the likelihood and the priors.

```r
bayesian_loc <- function(data, stan_file = "extreme_temperature.stan",
    iterations = 3000, chains = 1, refresh = 0) {

    results_list <- list()

    # Extract unique locations from the dataset
    unique_locations <- unique(data$location_name)

    for (location in unique_locations) {

        # Filter data for the current location
        location_data <- data %>%
            filter(location_name == !!location)

        data_for_stan <- list(N = nrow(location_data), y = location_data$max_avg_temp)

        # Sample from the posterior distribution
        fit <- stan(file = stan_file, data = data_for_stan, iter = iterations,
            chains = chains, refresh = refresh)

        results_list[[location]] <- fit

        # Only print the first location's result for
        # preview
        if (location == unique_locations[1]) {
            cat("\nResults for location:", location, "\n")
            print(summary(fit))
        }
    }

    return(results_list)
}
```

```r
results <- bayesian_loc(extreme_avg_data)
```

The results show that Bayesian inference using RStan was successfully performed for parameters associated with the GEV distribution for the data. The model appears to have converged well, with the posterior distributions for the parameters providing both point estimates and measures of uncertainty.

Posterior Summary: Means with Confidence Intervals

To evaluate the consistency and reliability of our Bayesian model across different locations, we summarize the posterior distributions of the parameters. Specifically, for each location, we extract the posterior means of the parameters (μ, σ, and ξ) along with their 95% confidence intervals.

```r
# Function to extract posterior means and CIs from stanfit
# object
extract_summary <- function(fit, params = c("loc", "scale", "shape")) {
    fit_summary <- summary(fit)$summary

    # Extracting the values for parameters
    sapply(params, function(param) {
        if (param %in% rownames(fit_summary)) {
            c(mean = fit_summary[param, "mean"], lower = fit_summary[param,
                "2.5%"], upper = fit_summary[param, "97.5%"])
        } else {
            c(mean = NA, lower = NA, upper = NA)
        }
    })
}

# Extract summaries for all locations
all_summaries <- lapply(results, extract_summary)

df_summary <- as.data.frame(do.call(rbind, lapply(all_summaries,
    function(x) {
        c(loc = sprintf("%.2f", x["mean", "loc"]), loc_lower = sprintf("%.2f",
            x["lower", "loc"]), loc_upper = sprintf("%.2f", x["upper",
            "loc"]), scale = sprintf("%.2f", x["mean", "scale"]),
            scale_lower = sprintf("%.2f", x["lower", "scale"]),
            scale_upper = sprintf("%.2f", x["upper", "scale"]),
            shape = sprintf("%.2f", x["mean", "shape"]), shape_lower = sprintf("%.2f",
                x["lower", "shape"]), shape_upper = sprintf("%.2f",
                x["upper", "shape"]))
    }))) %>%
    mutate(loc_interval = paste("(", loc_lower, ", ", loc_upper,
        ")", sep = ""), scale_interval = paste("(", scale_lower,
        ", ", scale_upper, ")", sep = ""), shape_interval = paste("(",
        shape_lower, ", ", shape_upper, ")", sep = "")) %>%
    select(loc, loc_interval, scale, scale_interval, shape, shape_interval)

df_summary
```

```r
df <- do.call(rbind, lapply(names(all_summaries), function(loc_name) {
    temp <- data.frame(all_summaries[[loc_name]])
    temp$location_name <- loc_name
    return(temp)
}))

df_long <- df %>%
    gather(key = "parameter", value = "value", -location_name)

# for (param in colnames(df)[1:3]) { print( ggplot(df,
# aes_string(x='location_name', y=param)) + geom_boxplot()
```

```r
# + ggtitle(paste('Boxplot for', param, 'parameter')) +
# labs(x = 'Location', y = 'Values') + theme_minimal() +
# theme(axis.text.x = element_text(angle = 45, hjust = 1))
# ) }

ggplot(df_long, aes(x = location_name, y = value)) + geom_boxplot() +
    facet_grid(parameter ~ ., scales = "free_y") + labs(x = "Location",
    y = "Values") + theme_minimal() + theme(axis.text.x = element_text(angle = 45,
    hjust = 1))
```

Posterior Analysis of the 100-Year Return Levels for Different Locations

Having obtained the posterior samples of our parameters from the GEV distribution using RStan, it is crucial to analyze these samples, especially when evaluating potential return levels. The 100-year return level, for instance, provides an estimate of the extreme value expected once every 100 years.

The formula for the 100-year return level, given the GEV parameters μ, σ and ξ is:

$$RL = \mu + \frac{\sigma}{\xi}\left(\left(\frac{1}{1-p}\right)^{\xi} - 1\right),$$

where p is the exceedance probability.

To compute the return level:

```r
# Function to calculate the return level
return_level <- function(loc, scale, shape) {
    p <- 1/100
    loc + (scale/shape) * ((1/(1 - p))^shape - 1)
}
```

```r
plot_df <- data.frame()

# For each location, calculate return levels
for (location_name in names(results)) {
    loc_samples <- rstan::extract(results[[location_name]])$loc
    scale_samples <- rstan::extract(results[[location_name]])$scale
    shape_samples <- rstan::extract(results[[location_name]])$shape

    RL_samples <- return_level(loc_samples, scale_samples, shape_samples)
    plot_df <- rbind(plot_df, data.frame(Location = location_name,
        ReturnLevel = RL_samples))
}

ggplot(plot_df, aes(x = ReturnLevel, color = Location)) + geom_density(aes(fill = Location),
    alpha = 0.4) + labs(title = "Posterior Density of 100-Year Return Levels",
    x = "100-Year Return Level", y = "Density") + theme_minimal() +
    theme(legend.position = "bottom")
```

Boxplots offer a visual insight into the dispersion, skewness, and potential outliers of the 100-year return levels for each location.

```r
ggplot(plot_df, aes(x = Location, y = ReturnLevel, fill = Location)) +
    geom_boxplot() + labs(title = "Boxplot of 100-Year Return Levels Across Locations",
    x = "Location", y = "100-Year Return Level") + theme_minimal() +
```

```r
    theme(legend.position = "none", axis.text.x = element_text(angle = 45,
        hjust = 1))
```

Extended Bayesian Inference for Different Models of `location`

In this extended section, we explore four different models for the location parameter μ in the GEV distribution. The models are:

1. Fixed μ;
2. Linear Trend: $\mu_t = \beta_0 + \beta t$;
3. Quadratic Trend: $\mu_t = \beta_0 + \beta_1 t + \beta_2 t^2$;

Model 1: Fixed μ

The existing model already assumes a fixed μ, represented by the `loc` parameter in the Stan code. No changes are needed for this model.

Model 2: Linear Trend: $\mu_t = \beta_0 + \beta t$

In this model, we introduce a linear trend in μ over time. The Stan code is modified to include β_0 and β as parameters.

```
data {
  int<lower=0> N;
  vector[N] y;
  vector[N] year;
}

parameters {
  real beta_0;
  real beta;
  // ... (other parameters)
}

transformed parameters {
  vector[N] mu_t;
  for (i in 1:N) {
    mu_t[i] = beta_0 + beta * year[i];
  }
}
```

Model 3: Quadratic Trend: $\mu_t = \beta_0 + \beta_1 t + \beta_2 t^2$

We add a quadratic term to capture more complex trends in μ over time.

```
parameters {
  real beta_0;
  real beta_1;
  real beta_2;
  // ... (other parameters)
}

transformed parameters {
  vector[N] mu_t;
  for (i in 1:N) {
    mu_t[i] = beta_0 + beta_1 * year[i] + beta_2 * pow(year[i], 2);
```

```
  }
}
```

Additional term `log_lik`

The `log_lik` term in the Stan model represents the log-likelihood of observing the data given the parameters. It is crucial for model comparison metrics like AIC.

```
generated quantities {
  vector[N] log_lik;
  for (n in 1:N) {
    log_lik[n] = gev_lpdf(y[n] | mu_t[n], scale, shape);
  }
}
```

R Code for Running Different Models

We modify the existing `bayesian_loc` function to accept an argument specifying the model type.

```
bayesian_loc_ex <- function(data, stan_file = "extreme_temperature_ex.stan",
    model_type, iterations = 2000, chains = 1, refresh = 0) {

    results_list <- list()

    # Extract unique locations from the dataset
    unique_locations <- unique(data$location_name)

    for (location in unique_locations) {

        # Filter data for the current location
        location_data <- data %>%
            filter(location_name == !!location)
        location_data$year <- location_data$year - min(location_data$year)
        data_for_stan <- list(N = nrow(location_data), y = location_data$max_avg_temp,
            year = location_data$year, model_type = model_type)

        # Sample from the posterior distribution
        fit <- stan(file = stan_file, data = data_for_stan, iter = iterations,
            chains = chains, refresh = refresh)

        results_list[[location]] <- fit

        # # Only print the first location's result for
        # preview if (location == unique_locations[1]) {
        # cat('\nResults for location:', location, '\n')
        # print(summary(fit)) }
    }

    return(results_list)
}
```

```
if (file.exists("results.RData")) {
    load("results.RData")
} else {
    results_1 <- bayesian_loc_ex(data = extreme_avg_data, model_type = 1)
```

```r
    results_2 <- bayesian_loc_ex(data = extreme_avg_data, model_type = 2)
    results_3 <- bayesian_loc_ex(data = extreme_avg_data, model_type = 3)
    save(results_1, results_2, results_3, file = "results.RData")
}
```

Model Comparison Across Locations Using AIC

For the Stan model, the AIC needs to be calculated manually, and the formula is:

$$AIC = -2 \times log - likelihood + 2 \times k$$

By comparing the AIC values, we can determine which model provides the best fit for each location while penalizing for complexity. This approach allows us to select the most appropriate model for each location, aiding in more accurate and interpretable results.

```r
aic_summary <- data.frame(Location = character(), Best_Model = character(),
    Model_1_AIC = numeric(), Model_2_AIC = numeric(), Model_3_AIC = numeric(),
    stringsAsFactors = FALSE)

extract_log_lik <- function(fit) {
    return(as.vector(rstan::extract(fit, "log_lik")[[1]]))
}

for (location in names(results_1)) {

    loglik1 <- sum(extract_log_lik(results_1[[location]]))
    loglik2 <- sum(extract_log_lik(results_2[[location]]))
    loglik3 <- sum(extract_log_lik(results_3[[location]]))

    aic1 <- format(-2 * loglik1 + 2 * 3, scientific = FALSE)  # 3 parameters for Model 1
    aic2 <- format(-2 * loglik2 + 2 * 4, scientific = FALSE)  # 4 parameters for Model 2
    aic3 <- format(-2 * loglik3 + 2 * 5, scientific = FALSE)  # 5 parameters for Model 3

    best_model <- which.min(c(aic1, aic2, aic3))

    summary_row <- data.frame(Location = location, Best_Model = paste("Model",
        best_model), Model_1_AIC = aic1, Model_2_AIC = aic2,
        Model_3_AIC = aic3, stringsAsFactors = FALSE)

    aic_summary <- rbind(aic_summary, summary_row)
}

aic_summary
```

Location	Best_Model	Model_1_AIC	Model_2_AIC	Model_3_AIC
Auckland	Model 2	139102	133143.5	134096
Blenheim	Model 1	165432.3	166535.1	165631
Christchurch	Model 1	225552.8	226606.8	225862.5
Dannevirke	Model 3	217189.9	210342.2	207077.9
Dunedin	Model 1	263355.6	264222.8	264824.5
Gisborne	Model 3	281654.8	276734.8	275689.2
Gore	Model 3	157742.1	158411.9	150502
Hamilton	Model 2	134989.6	128887.5	129818.7

Location	Best_Model	Model_1_AIC	Model_2_AIC	Model_3_AIC
Hokitika	Model 1	180712.7	181782.5	181248.6
Invercargill	Model 2	240029.2	235736.9	241364.2
Kerikeri	Model 1	116455.3	116524.5	117563.6
Lake Tekapo	Model 3	331692.3	328469.1	328402.3
Masterton	Model 2	169187.2	162263.8	170648.5
Milford Sound	Model 2	261486.1	260144.6	261821.6
Napier	Model 3	253313.6	250345	248961.7
Nelson	Model 3	267419.3	257197.3	256521.8
New Plymouth	Model 2	218325	194751.2	198745.5
Queenstown	Model 3	162298.9	154792.2	154265.5
Reefton	Model 3	163895.3	156884.7	149400.9
Rotorua	Model 3	132110.2	132037.7	130957.2
Tara Hills	Model 2	253312.4	250931.7	251756.9
Taumarunui	Model 1	211840.5	212690.8	213385
Taupo	Model 2	141328	139548.9	140052.3
Tauranga	Model 3	217345.2	212760	209309.5
Timaru	Model 1	180415.8	180604.1	181412.2
Waiouru	Model 2	139617.7	129052.2	130047.7
Wellington	Model 2	281787.4	277533.7	278557.4
Whanganui	Model 2	141408.6	140733.7	142342.9
Whangaparaoa	Model 3	116407	105692.2	101574
Whangarei	Model 2	149367.8	143390.5	143990.3

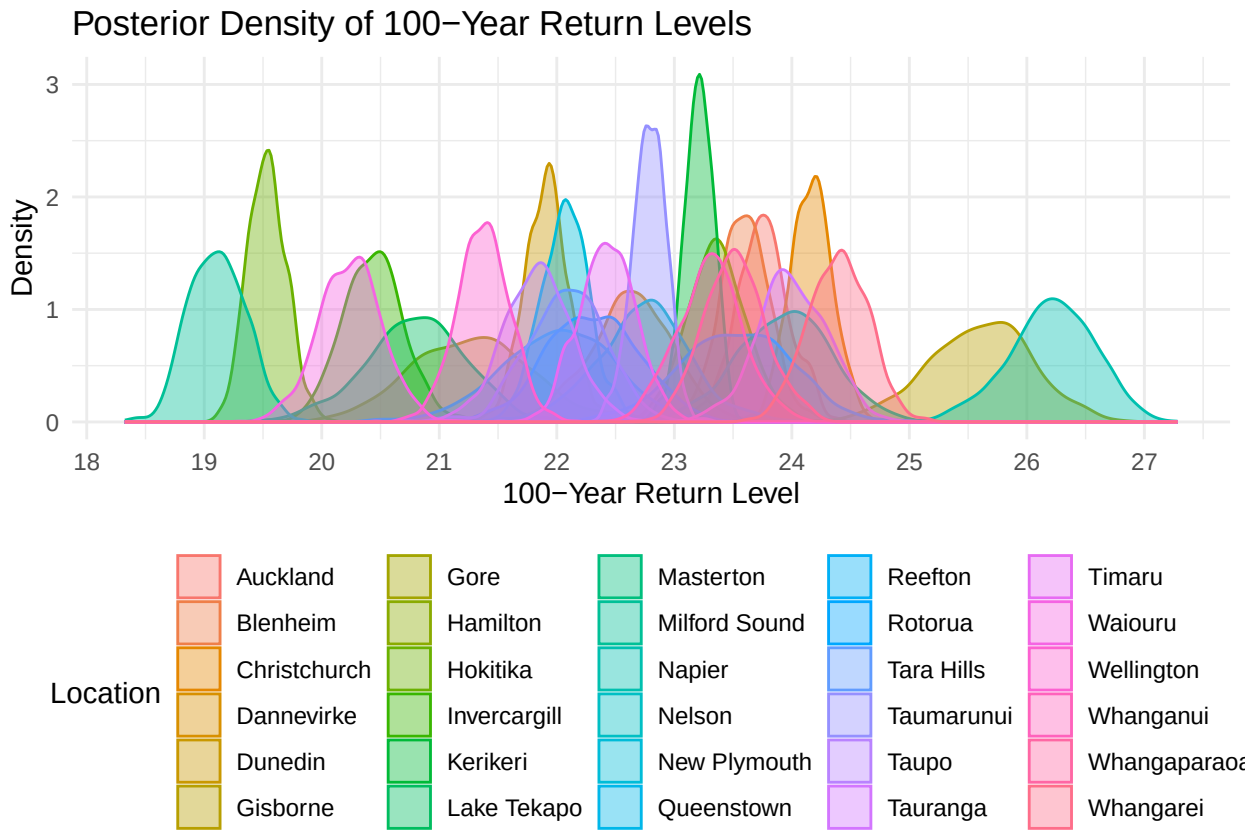
```r
# store best models
best_models <- list()

for (i in 1:nrow(aic_summary)) {
    location <- aic_summary$Location[i]

    aic_values <- c(Model_1_AIC = aic_summary$Model_1_AIC[i],
        Model_2_AIC = aic_summary$Model_2_AIC[i], Model_3_AIC = aic_summary$Model_3_AIC[i])

    best_model_num <- which.min(aic_values)
    best_model_fit <- get(paste("results_", best_model_num, sep = ""))[[location]]
    best_models[[location]] <- list(model_num = best_model_num,
        fit = best_model_fit)
}
```

100 Year Return Level

```r
return_level_df <- data.frame(Location = character(), ReturnLevel = numeric(),
    stringsAsFactors = FALSE)

for (location in names(best_models)) {

    fit <- best_models[[location]]$fit

    loc_t_samples <- rstan::extract(fit)$loc_t
    loc_samples <- loc_t_samples[, ncol(loc_t_samples)]
```

```r
    scale_samples <- rstan::extract(fit)$scale
    shape_samples <- rstan::extract(fit)$shape

    RL_samples <- return_level(loc_samples, scale_samples, shape_samples)

    return_level_df <- rbind(return_level_df, data.frame(Location = location,
        ReturnLevel = RL_samples))
}
```

```r
ggplot(return_level_df, aes(x = ReturnLevel, color = Location)) +
    geom_density(aes(fill = Location), alpha = 0.4) + labs(title = "Posterior Density of 100-Y
    x = "100-Year Return Level", y = "Density") + theme_minimal() +
    theme(legend.position = "bottom") + scale_x_continuous(breaks = scales::pretty_breaks(n =
```

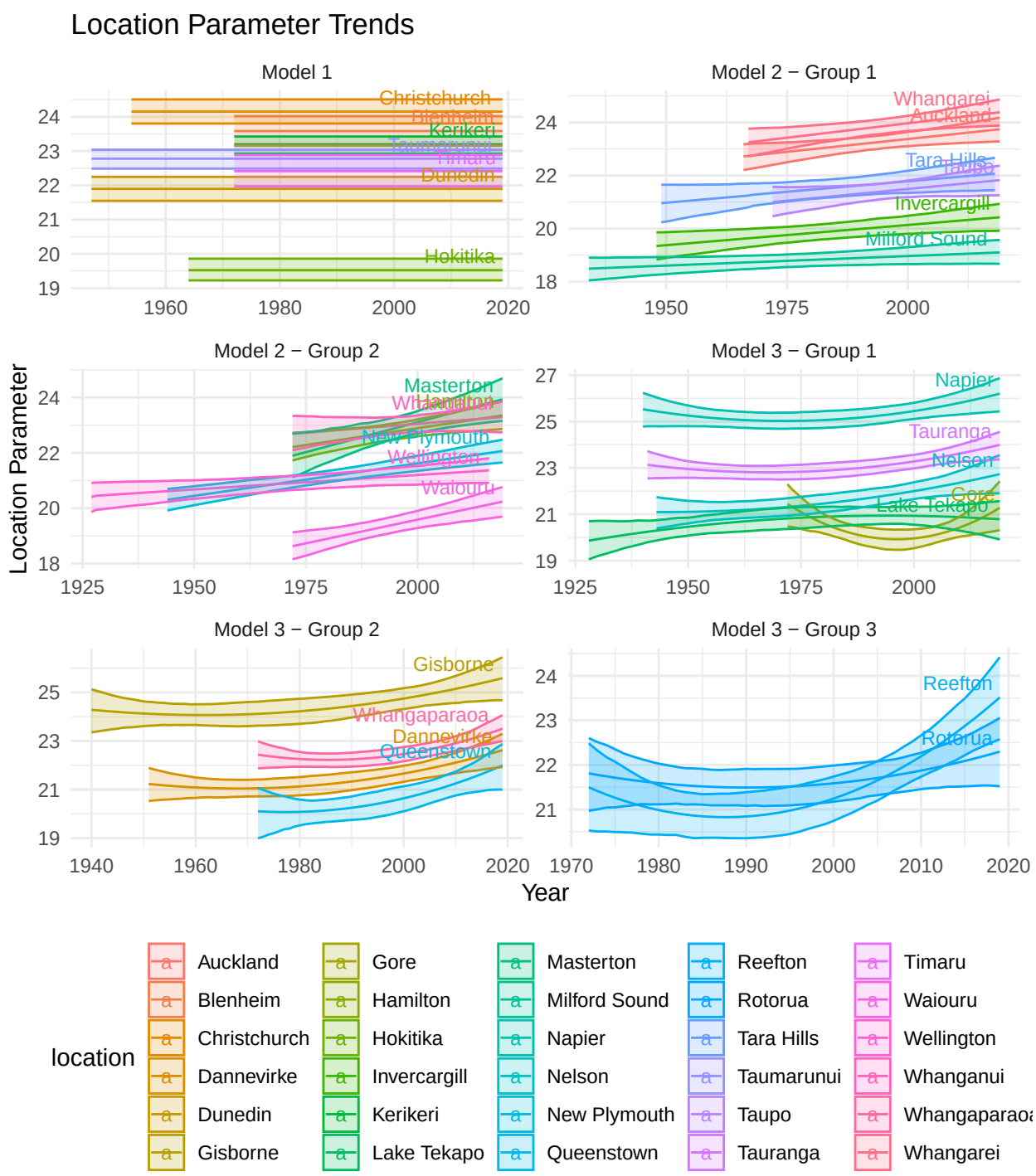


Analysis and Visualization of Location Parameter Trends across Different Locations and Models

```r
location_trends <- data.frame(year = integer(), location = character(),
    model = character(), mean_loc = numeric(), lower_loc = numeric(),
    upper_loc = numeric(), stringsAsFactors = FALSE)

group2_1 <- c("Whangarei", "Auckland", "Tara Hills", "Taupo",
    "Invercargill", "Milford Sound")
group2_2 <- c("Masterton", "Whanganui", "New Plymouth", "Wellington",
    "Waiouru", "Hamilton")
group3_1 <- c("Napier", "Tauranga", "Nelson", "Gore", "Lake Tekapo")
group3_2 <- c("Gisborne", "Whangaparaoa", "Dannevirke", "Queenstown")
group3_3 <- c("Rotorua", "Reefton")
```

```r
for (location in names(best_models)) {
    best_model_num <- best_models[[location]]$model_num
    fit <- best_models[[location]]$fit
    if (location %in% group2_1) {
        model_name <- c("Model 2 - Group 1")
    } else if (location %in% group2_2) {
        model_name <- c("Model 2 - Group 2")
    } else if (location %in% group3_1) {
        model_name <- c("Model 3 - Group 1")
    } else if (location %in% group3_2) {
        model_name <- c("Model 3 - Group 2")
    } else if (location %in% group3_3) {
        model_name <- c("Model 3 - Group 3")
    } else {
        model_name <- paste("Model", best_model_num)
    }

    loc_t_samples <- rstan::extract(fit)$loc_t

    min_year <- min(extreme_avg_data$year[extreme_avg_data$location_name ==
        location])

    for (i in seq_len(ncol(loc_t_samples))) {
        year_loc_samples <- loc_t_samples[, i]
        year_mean <- mean(year_loc_samples)
        year_lower <- quantile(year_loc_samples, 0.025)
        year_upper <- quantile(year_loc_samples, 0.975)

        real_year <- min_year + i - 1

        location_trends <- rbind(location_trends, data.frame(year = real_year,
            location = location, model = model_name, mean_loc = year_mean,
            lower_loc = year_lower, upper_loc = year_upper))
    }
}
```

```r
p <- ggplot(location_trends, aes(x = year, y = mean_loc, color = location,
    fill = location)) + geom_line() + geom_ribbon(aes(ymin = lower_loc,
    ymax = upper_loc), alpha = 0.2) + labs(title = "Location Parameter Trends",
    x = "Year", y = "Location Parameter") + theme_minimal() +
    theme(legend.position = "bottom") + facet_wrap(~model, scales = "free",
    ncol = 2)

last_year_data <- location_trends %>%
    group_by(location, model) %>%
    filter(year == max(year))

p <- p + geom_text(data = last_year_data, aes(label = location),
    vjust = -0.5, hjust = 1.1, size = 3, angle = 0)

print(p)
```

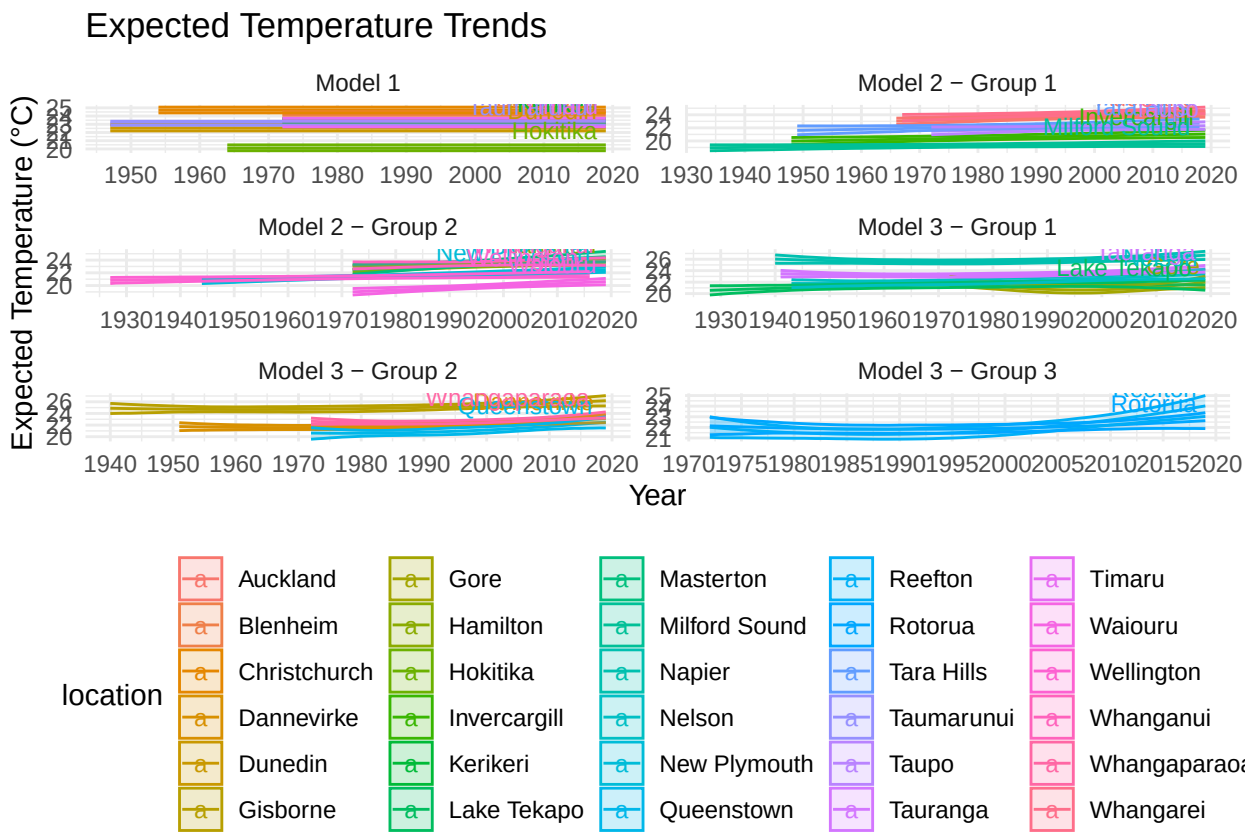


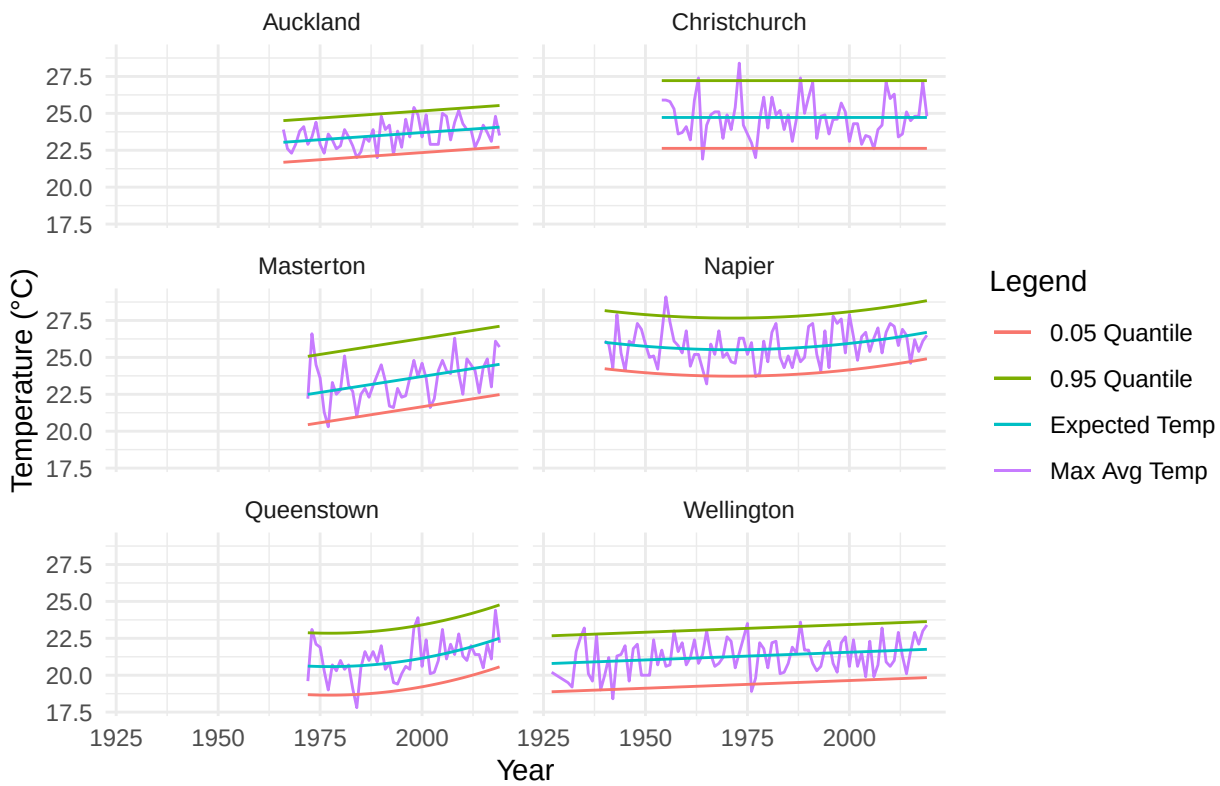
Expected Temperature Trends Across Locations

```
expected_temp_trends <- data.frame(year = integer(), location = character(),
    ExpectedTemp = numeric(), LowerTemp = numeric(), UpperTemp = numeric(),
    Model = numeric(), stringsAsFactors = FALSE)

for (location in names(best_models)) {

    best_model_num <- best_models[[location]]$model_num
    fit <- best_models[[location]]$fit
    if (location %in% group2_1) {
        model_name <- c("Model 2 - Group 1")
```

```r
    } else if (location %in% group2_2) {
        model_name <- c("Model 2 - Group 2")
    } else if (location %in% group3_1) {
        model_name <- c("Model 3 - Group 1")
    } else if (location %in% group3_2) {
        model_name <- c("Model 3 - Group 2")
    } else if (location %in% group3_3) {
        model_name <- c("Model 3 - Group 3")
    } else {
        model_name <- paste("Model", best_model_num)
    }

    loc_t_samples <- rstan::extract(fit)$loc_t
    scale_samples <- rstan::extract(fit)$scale
    shape_samples <- rstan::extract(fit)$shape

    min_year <- min(extreme_avg_data$year[extreme_avg_data$location_name ==
        location])

    for (i in seq_len(ncol(loc_t_samples))) {
        year_loc_samples <- loc_t_samples[, i]

        year_expected_temp <- year_loc_samples + scale_samples *
            (gamma(1 - shape_samples) - 1)/shape_samples
        expected_mean <- mean(year_expected_temp)
        expected_lower <- quantile(year_expected_temp, 0.025)
        expected_upper <- quantile(year_expected_temp, 0.975)

        real_year <- min_year + i - 1

        expected_temp_trends <- rbind(expected_temp_trends, data.frame(year = real_year,
            location = location, model = model_name, ExpectedTemp = expected_mean,
            LowerTemp = expected_lower, UpperTemp = expected_upper))
    }
}
```

```r
p_temp <- ggplot(expected_temp_trends, aes(x = year, y = ExpectedTemp,
    color = location, fill = location)) + geom_line() + geom_ribbon(aes(ymin = LowerTemp,
    ymax = UpperTemp), alpha = 0.2) + labs(title = "Expected Temperature Trends",
    x = "Year", y = "Expected Temperature (°C)") + theme_minimal() +
    theme(legend.position = "bottom") + scale_x_continuous(breaks = scales::pretty_breaks(n =
    facet_wrap(~model, scales = "free", ncol = 2)

last_year_temp_data <- expected_temp_trends %>%
    group_by(location) %>%
    filter(year == max(year))

p_temp <- p_temp + geom_text(data = last_year_temp_data, aes(label = location),
    vjust = -0.5, hjust = 1.1, size = 3, angle = 0)

print(p_temp)
```




```
get_quantiles_for_location <- function(location, extreme_avg_data,
    best_models, expected_temp_trends) {
    Loc = rstan::extract(best_models[[location]]$fit)$loc_t
    Shape = rstan::extract(best_models[[location]]$fit)$shape
    Scale = rstan::extract(best_models[[location]]$fit)$scale

    q0 = q1 = matrix(0, nrow(Loc), ncol(Loc))
    for (t in 1:ncol(Loc)) {
        for (i in 1:nrow(Loc)) {
            q0[i, t] = qgev(0.05, Loc[i, t], Scale[i], Shape[i])
            q1[i, t] = qgev(0.95, Loc[i, t], Scale[i], Shape[i])
        }
    }

    data = data.frame(year = extreme_avg_data$year, max_avg_temp = extreme_avg_data$max_avg_te
        expected_temp = expected_temp_trends$ExpectedTemp, q0 = colMeans(q0),
        q1 = colMeans(q1), location = location)

    return(data)
}

locations = c("Auckland", "Napier", "Wellington", "Masterton",
    "Christchurch", "Queenstown")

all_data = data.frame()

for (loc in locations) {
    extreme_avg_data_loc = extreme_avg_data %>%
        filter(location_name == loc)
```

```r
    expected_temp_trends_loc = expected_temp_trends %>%
        filter(location == loc)
    data = get_quantiles_for_location(loc, extreme_avg_data_loc,
        best_models, expected_temp_trends_loc)

    all_data = rbind(all_data, data)
}

plot = ggplot(all_data, aes(x = year)) + labs(title = "", x = "Year",
    y = "Temperature (°C)", color = "Location") + geom_line(aes(y = max_avg_temp,
    color = "Max Avg Temp")) + geom_line(aes(y = expected_temp,
    color = "Expected Temp")) + geom_line(aes(y = q0, color = "0.05 Quantile")) +
    geom_line(aes(y = q1, color = "0.95 Quantile")) + facet_wrap(~location,
    ncol = 2) + theme_minimal() + labs(color = "Legend")

print(plot)
```